

Computer Networks Interview Guide

1. Overview

Computer networks are systems that allow devices to communicate and exchange data. When you open a website, send a message, stream a video, or call an API from backend code, computer networking concepts decide how that data moves from one machine to another.

Definition

A computer network is a collection of connected devices that communicate using standard protocols such as HTTP, TCP, UDP, DNS, IP, and TLS.

Why it matters

Networking is the foundation of almost every real-world software system:

- Web applications depend on HTTP, HTTPS, DNS, cookies, sessions, and TLS.
- Backend services communicate using sockets, TCP, UDP, load balancers, and reverse proxies.
- Distributed systems rely on reliable communication, routing, fault tolerance, and secure data transfer.
- Cloud platforms use NAT, subnetting, private networks, gateways, and load balancers.

Where it is used in real systems

- Browser requesting a webpage from a server
- Mobile app calling backend APIs
- Backend service connecting to a database
- DNS resolving `google.com` to an IP address
- HTTPS protecting login credentials
- Load balancer distributing traffic across multiple servers
- NAT allowing private devices to access the internet

Why interviewers ask about it

Interviewers ask networking questions because SDEs must understand how applications communicate in real systems. These questions test whether you can reason about latency, reliability, security, APIs, authentication, and debugging production issues.

Common placement focus areas:

- What happens when you open a URL?
- HTTP vs HTTPS
- GET vs POST
- TCP vs UDP
- TCP 3-way handshake
- DNS resolution
- Cookies and sessions
- 401 vs 403

- TLS handshake
- Load balancer vs reverse proxy

2. Core Idea

The core idea of computer networks is simple:

Data is broken into smaller units, sent through multiple layers and devices, and reconstructed at the destination using agreed rules called protocols.

Intuition

Imagine sending a parcel from one city to another:

- You write the address.
- The courier company chooses a route.
- The parcel may pass through many hubs.
- The receiver confirms delivery.
- If the parcel is important, tracking and verification are used.

Networking works similarly:

- IP address identifies the destination machine.
- DNS converts domain names to IP addresses.
- TCP or UDP decides how data is transported.
- HTTP defines how web clients and servers talk.
- TLS secures the communication.
- Routers, NAT, load balancers, and proxies help move traffic.

Real-world analogy

Networking Concept	Real-World Analogy
IP address	House address
DNS	Phonebook/contact list
TCP	Registered courier with delivery confirmation
UDP	Fast postcard without confirmation
HTTP	Language used between client and server
HTTPS	Same language, but inside a locked envelope
Cookie	ID card stored with the client
Session	Server-side memory about a user
Load balancer	Reception desk distributing visitors
Reverse proxy	Front office that talks to internal departments

Small example

When you open:

```
https://example.com/login
```

The browser roughly does this:

1. Uses DNS to find the IP address of `example.com`.
2. Opens a TCP connection to the server.
3. Performs a TLS handshake because the URL uses HTTPS.
4. Sends an HTTP request such as `GET /login`.
5. Server sends back an HTTP response.
6. Browser renders the page.
7. If you log in, the server may send a cookie containing a session ID.

3. Important Subtopics

3.1 HTTP vs HTTPS

What it means

HTTP stands for HyperText Transfer Protocol. It is the protocol used by browsers and servers to exchange web data.

HTTPS is HTTP over TLS. It encrypts the communication between client and server.

Why it matters

HTTP sends data in plain text. HTTPS protects sensitive data such as passwords, tokens, cookies, and payment details.

Example

```
http://example.com  
https://example.com
```

With HTTP, someone on the network may read or modify traffic. With HTTPS, data is encrypted and the server identity is verified using certificates.

Common interview angle

Interviewers often ask:

- Is HTTPS a different protocol from HTTP?
- What security does HTTPS provide?
- What happens during the TLS handshake?

3.2 GET vs POST

What it means

GET and POST are HTTP methods.

GET is usually used to fetch data. POST is usually used to submit data or create something.

Why it matters

Choosing the correct HTTP method affects API design, caching, browser behavior, security, and idempotency.

Example

```
GET /products
POST /orders
```

GET /products fetches products.

POST /orders creates a new order.

Common interview angle

Interviewers expect you to know that GET is safe and usually idempotent, while POST is not necessarily idempotent.

3.3 401 vs 403

What it means

Both are HTTP status codes related to access control.

- **401 Unauthorized**: Authentication is missing or invalid.
- **403 Forbidden**: Authentication may be valid, but the user does not have permission.

Why it matters

APIs must return correct status codes so clients can handle login, authorization, and permission errors correctly.

Example

```
401: You are not logged in.
403: You are logged in, but you are not an admin.
```

Common interview angle

Many students confuse authentication and authorization. Interviewers use 401 vs 403 to test this difference.

3.4 Cookie vs Session

What it means

A cookie is small data stored in the browser.

A session is user-specific data usually stored on the server.

Why it matters

HTTP is stateless. Cookies and sessions help servers remember users across multiple requests.

Example

After login:

```
Server creates session: session_id = abc123
Server sends cookie: session_id=abc123
Browser sends this cookie on future requests
Server uses abc123 to find user data
```

Common interview angle

Interviewers ask whether session data is stored on client or server, and how cookies help maintain login state.

3.5 TCP vs UDP

What it means

TCP and UDP are transport layer protocols.

TCP provides reliable, ordered, connection-based communication.

UDP provides faster, connectionless communication without guaranteed delivery.

Why it matters

Different applications have different needs. File transfer needs reliability. Video calls need speed and can tolerate some packet loss.

Example

- TCP: HTTP, HTTPS, SSH, FTP
- UDP: DNS, video calls, online gaming, live streaming

Common interview angle

Interviewers ask which protocol to choose for a use case and why.

3.6 TCP 3-Way Handshake

What it means

The TCP 3-way handshake establishes a connection between client and server before data transfer.

Why it matters

It ensures both sides are ready and agree on initial sequence numbers.

Example

```
Client -> Server: SYN  
Server -> Client: SYN-ACK  
Client -> Server: ACK
```

After this, the TCP connection is established.

Common interview angle

Interviewers ask why three steps are needed instead of two.

3.7 Why TCP Is Reliable

What it means

TCP reliability means data is delivered correctly, in order, and without missing parts, as long as the connection does not fail completely.

Why it matters

Applications like web pages, banking, file transfer, and email cannot tolerate corrupted or missing data.

How TCP provides reliability

- Sequence numbers
- Acknowledgements
- Retransmission of lost packets
- Checksums
- Flow control
- Congestion control
- Ordered delivery

Common interview angle

Interviewers ask how TCP handles packet loss.

3.8 DNS

What it means

DNS stands for Domain Name System. It converts human-readable domain names into IP addresses.

Why it matters

Humans remember names like `google.com`; computers communicate using IP addresses.

Example

```
www.google.com -> 142.250.192.4
```

Common interview angle

Interviewers ask what happens during DNS resolution when you open a URL.

3.9 NAT

What it means

NAT stands for Network Address Translation. It allows devices using private IP addresses to communicate with the internet using a public IP address.

Why it matters

There are not enough IPv4 addresses for every device to have a unique public IP. NAT helps conserve public IPs and hides internal network details.

Example

```
Laptop: 192.168.1.10  
Phone: 192.168.1.11  
Router public IP: 49.36.20.100
```

Both laptop and phone access the internet through the router's public IP.

Common interview angle

Interviewers ask how multiple private devices share one public IP.

3.10 Subnetting

What it means

Subnetting divides a larger network into smaller logical networks called subnets.

Why it matters

Subnetting improves organization, security, routing, and IP address management.

Example

```
192.168.1.0/24
```

This means:

- Network: 192.168.1.0
- Usable host range: usually 192.168.1.1 to 192.168.1.254
- Broadcast: 192.168.1.255
- Total addresses: 256

Common interview angle

Interviewers ask how many hosts are possible in a subnet such as /24, /16, or /30.

3.11 Sockets

What it means

A socket is an endpoint for communication between two machines or processes.

Why it matters

Sockets are the programming interface used to build network applications.

Example

```
Client socket: 192.168.1.5:53000  
Server socket: 93.184.216.34:443
```

A socket is usually identified by IP address plus port number.

Common interview angle

Interviewers ask what a socket is and how client-server communication happens.

3.12 TLS Handshake

What it means

The TLS handshake establishes a secure encrypted connection between client and server.

Why it matters

It protects data confidentiality, verifies server identity, and prevents tampering.

Simplified steps

```
ClientHello  
ServerHello + Certificate  
Certificate verification  
Key exchange  
Session keys created  
Encrypted communication starts
```

Common interview angle

Interviewers ask how HTTPS is secure and what role certificates play.

3.13 Load Balancer vs Reverse Proxy

What it means

A load balancer distributes traffic across multiple backend servers.

A reverse proxy receives client requests and forwards them to internal servers. It may also do caching, SSL termination, compression, routing, and security filtering.

Why it matters

Modern systems need scalability, availability, security, and centralized traffic control.

Example

```
Client -> Load Balancer -> Server 1 / Server 2 / Server 3  
Client -> Reverse Proxy -> Internal App Server
```

Common interview angle

Interviewers ask whether a reverse proxy and load balancer are the same. They overlap, but their main purposes are different.

4. Real-World Example

What happens when you open a website?

Suppose you open:

```
https://shop.example.com/products
```

Step-by-step:

1. Browser checks cache for DNS result.
2. If not cached, browser asks DNS resolver for the IP address.
3. DNS resolver returns an IP address.
4. Browser opens a TCP connection to the server on port 443.
5. TCP 3-way handshake happens.
6. TLS handshake happens to create a secure HTTPS connection.
7. Browser sends HTTP request:

```
GET /products HTTP/1.1
Host: shop.example.com
Cookie: session_id=abc123
```

8. Request may first reach a load balancer.
9. Load balancer forwards the request to one backend server.
10. Reverse proxy may route request to the correct internal service.
11. Backend server reads cookie/session data.
12. Server generates response.
13. Response travels back through TCP.
14. Browser receives HTML, CSS, JS, and images.
15. Browser renders the page.

This single action uses DNS, TCP, TLS, HTTPS, cookies, sessions, load balancers, reverse proxies, sockets, NAT, and subnetting.

5. Diagrams / Mental Models

URL opening flow

```
User enters URL
  |
  v
DNS resolves domain to IP
  |
  v
TCP 3-way handshake
  |
  v
TLS handshake for HTTPS
  |
  v
```

```

HTTP request sent
  |
  v
Load balancer / reverse proxy
  |
  v
Backend server
  |
  v
HTTP response
  |
  v
Browser renders page

```

TCP 3-way handshake

```

Client                                Server
|                                     |
| SYN                                |
|----->|                             |
|                                     |
| SYN-ACK                            |
|<-----|                             |
|                                     |
| ACK                                |
|----->|                             |
|                                     |
| Connection established              |
|                                     |

```

Cookie and session model

1. User logs in
2. Server creates session data
3. Server sends session_id cookie
4. Browser stores cookie
5. Browser sends cookie on future requests
6. Server finds session using session_id

NAT mental model

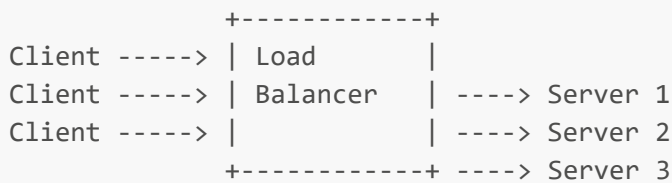
Private Network

```

Laptop 192.168.1.10 ----\
Phone  192.168.1.11 ----- Router/NAT ---- Internet
TV      192.168.1.12 ----/ Public IP: 49.36.20.100

```

Load balancer model



6. Common Interview Questions

1. What is the difference between HTTP and HTTPS?

HTTP is a protocol for communication between client and server. HTTPS is HTTP secured using TLS encryption.

Key points interviewer expects:

- HTTPS encrypts data.
- HTTPS verifies server identity using certificates.
- HTTPS protects against eavesdropping and tampering.
- HTTPS usually uses port **443**; HTTP usually uses port **80**.

Common mistakes:

- Saying HTTPS is only "faster" than HTTP.
- Saying HTTPS means the website is completely safe.
- Forgetting the role of TLS certificates.

2. What is the difference between GET and POST?

GET is mainly used to retrieve data. POST is mainly used to submit data or create/update resources.

Key points interviewer expects:

- GET parameters are often visible in URL.
- POST usually sends data in request body.
- GET is safe and usually idempotent.
- POST is not necessarily idempotent.
- GET can be cached more easily.

Common mistakes:

- Saying GET cannot have a body as an absolute rule.
- Saying POST is always secure. POST is not secure unless HTTPS is used.
- Saying GET is only for small data without mentioning semantics.

3. What is the difference between 401 and 403?

401 Unauthorized means authentication is missing or invalid. **403 Forbidden** means the user is authenticated but does not have permission.

Key points interviewer expects:

- 401 is about identity.
- 403 is about permission.
- Authentication and authorization are different.

Common mistakes:

- Saying both mean "access denied" without distinction.
- Saying 403 means user is not logged in.

4. What is the difference between cookie and session?

A cookie is stored on the client/browser. A session is usually stored on the server. Cookies often store a session ID used to identify server-side session data.

Key points interviewer expects:

- HTTP is stateless.
- Cookie helps maintain state across requests.
- Session data is safer on server than storing all user data in browser.
- Cookies can have security flags like `HttpOnly`, `Secure`, and `SameSite`.

Common mistakes:

- Saying cookies and sessions are the same.
- Saying sessions are always stored in cookies.
- Storing sensitive data directly in cookies without protection.

5. What is the difference between TCP and UDP?

TCP is connection-oriented, reliable, ordered, and slower due to overhead. UDP is connectionless, faster, and does not guarantee delivery or ordering.

Key points interviewer expects:

- TCP has acknowledgements and retransmissions.
- UDP has lower latency.
- TCP is used for web, SSH, file transfer.
- UDP is used for DNS, gaming, streaming, VoIP.

Common mistakes:

- Saying UDP is always bad because it is unreliable.
- Saying TCP guarantees delivery even if the network completely fails.

6. Explain the TCP 3-way handshake.

The TCP 3-way handshake establishes a connection:

```
Client sends SYN
Server sends SYN-ACK
Client sends ACK
```

Key points interviewer expects:

- Both client and server confirm readiness.
- Initial sequence numbers are exchanged.
- Connection is established before data transfer.

Common mistakes:

- Forgetting sequence numbers.
- Saying data is always sent during the handshake.

7. Why is TCP reliable?

TCP is reliable because it uses sequence numbers, acknowledgements, retransmissions, checksums, flow control, congestion control, and ordered delivery.

Key points interviewer expects:

- Lost packets are retransmitted.
- Duplicate or out-of-order packets can be handled.
- Receiver acknowledges received data.
- Checksums detect corruption.

Common mistakes:

- Saying TCP is reliable only because it has a connection.
- Ignoring flow control and congestion control.

8. What is DNS?

DNS converts domain names into IP addresses. It allows users to access websites using names instead of numeric IP addresses.

Key points interviewer expects:

- DNS is hierarchical.
- Browser or OS may cache DNS results.
- DNS records include A, AAAA, CNAME, MX, TXT.
- DNS commonly uses UDP port 53, but can use TCP too.

Common mistakes:

- Saying DNS stores websites.
- Saying DNS always returns only one IP.

9. What is NAT?

NAT translates private IP addresses to a public IP address and vice versa. It allows multiple private devices to access the internet using one public IP.

Key points interviewer expects:

- Private IPs are not routable on the public internet.
- Router maintains a translation table.
- NAT conserves IPv4 addresses.

Common mistakes:

- Saying NAT is the same as a firewall.
- Forgetting port translation.

10. What is subnetting?

Subnetting divides a network into smaller networks. It is used for efficient IP allocation, routing, isolation, and management.

Key points interviewer expects:

- CIDR notation such as `/24`.
- Network address, broadcast address, and host range.
- Smaller subnets reduce broadcast domains.

Common mistakes:

- Confusing subnet mask with default gateway.
- Forgetting that some addresses are reserved.

11. What is a socket?

A socket is a communication endpoint identified by an IP address and port number. It allows applications to send and receive data over a network.

Key points interviewer expects:

- Socket is used in client-server programming.
- TCP sockets are connection-oriented.
- UDP sockets are connectionless.
- A connection is identified by source IP, source port, destination IP, destination port, and protocol.

Common mistakes:

- Saying socket means only port number.
- Confusing socket with physical cable connection.

12. What happens in a TLS handshake?

The TLS handshake allows client and server to agree on encryption parameters, verify server identity, and generate shared session keys.

Key points interviewer expects:

- ClientHello and ServerHello.
- Server sends certificate.
- Client verifies certificate.
- Key exchange happens.
- Symmetric session keys are used for actual data transfer.

Common mistakes:

- Saying all data is encrypted using the server's public key.
- Ignoring certificate verification.

13. Load balancer vs reverse proxy?

A load balancer distributes traffic across multiple backend servers. A reverse proxy sits in front of servers and forwards client requests to internal services.

Key points interviewer expects:

- Load balancer improves scalability and availability.
- Reverse proxy hides backend details.
- Reverse proxy can do SSL termination, caching, compression, and routing.
- Many tools can perform both roles.

Common mistakes:

- Saying they are always completely separate.
- Saying reverse proxy only works for static websites.

7. Deep-Dive Questions

1. Why does TCP need a 3-way handshake instead of a 2-way handshake?

TCP needs three steps so both sides can confirm that they can send and receive data. The server must know the client received its **SYN-ACK**; otherwise, half-open or stale connections may occur.

In short:

```
SYN: Client can send.  
SYN-ACK: Server can receive and send.  
ACK: Client can receive server's response.
```

2. How does TCP handle packet loss?

TCP detects packet loss using missing acknowledgements, duplicate acknowledgements, and timeouts. When loss is detected, TCP retransmits the missing data. Sequence numbers help the receiver place data in correct order.

3. Why does HTTPS still use symmetric encryption after the TLS handshake?

Public-key encryption is useful for authentication and key exchange, but it is computationally expensive. After the TLS handshake, both sides use symmetric session keys because symmetric encryption is much faster for large data transfer.

4. Can UDP be made reliable?

Yes. Reliability can be implemented at the application layer over UDP. For example, an application can add sequence numbers, acknowledgements, retransmissions, and ordering logic. Some modern protocols like QUIC use UDP but implement reliability and security above it.

5. How does a load balancer know which server to send a request to?

A load balancer can use algorithms such as:

- Round robin
- Least connections
- Weighted round robin
- IP hash
- Health-check based routing

It also performs health checks so traffic is not sent to unhealthy servers.

8. Comparison Tables

HTTP vs HTTPS

Feature	HTTP	HTTPS
Full form	HyperText Transfer Protocol	HTTP Secure
Security	No encryption	Encrypted using TLS
Default port	80	443
Certificate required	No	Yes
Protects credentials	No	Yes
Used for	Basic web communication	Secure web communication

GET vs POST

Feature	GET	POST
Purpose	Retrieve data	Submit/create data
Data location	Usually URL query string	Usually request body
Cacheable	More commonly cacheable	Usually not cached by default
Idempotent	Usually yes	Not necessarily
Bookmarkable	Yes	No

Feature	GET	POST
Example	GET /users	POST /users

401 vs 403

Feature	401 Unauthorized	403 Forbidden
Meaning	Not authenticated	Not authorized
Login required?	Usually yes	Login may already be done
Example	Missing token	Non-admin accessing admin page
Client action	Login or send valid credentials	Request permission or use allowed account

Cookie vs Session

Feature	Cookie	Session
Stored at	Client/browser	Usually server
Used for	Remembering small data or session ID	Storing user state
Security	Can be modified unless protected	More controlled by server
Lifetime	Controlled by expiry/max-age	Controlled by server/session store
Example	session_id=abc123	User ID, cart, login state

TCP vs UDP

Feature	TCP	UDP
Connection	Connection-oriented	Connectionless
Reliability	Reliable	Best effort
Ordering	Maintains order	No ordering guarantee
Speed	More overhead	Lower overhead
Acknowledgement	Yes	No by default
Use cases	Web, SSH, email, file transfer	DNS, gaming, VoIP, streaming

Load Balancer vs Reverse Proxy

Feature	Load Balancer	Reverse Proxy
Main purpose	Distribute traffic	Forward and manage requests
Backend count	Usually multiple servers	One or more servers
Common functions	Health checks, scaling, failover	Routing, caching, SSL termination

Feature	Load Balancer	Reverse Proxy
Client sees backend?	No	No
Example tools	AWS ELB, HAProxy, NGINX	NGINX, Apache, Envoy

9. Common Mistakes

- Confusing authentication with authorization.
- Saying POST is secure even without HTTPS.
- Saying HTTPS means a website is always trustworthy.
- Thinking cookies and sessions are the same thing.
- Forgetting that HTTP is stateless.
- Saying UDP is useless because it is unreliable.
- Saying TCP guarantees delivery under all conditions.
- Forgetting DNS caching.
- Confusing NAT with DNS.
- Confusing subnet mask with default gateway.
- Thinking a socket is only a port.
- Saying reverse proxy and load balancer are always completely different tools.
- Forgetting that DNS can use TCP as well as UDP.
- Thinking TLS only encrypts data and does not verify server identity.

10. Edge Cases / Special Cases

GET can technically have a body

HTTP does not strictly forbid a GET request body, but many servers, proxies, and clients do not handle it consistently. In interviews, say GET usually sends data through query parameters and should be used for retrieval.

POST is not automatically secure

POST data is not visible in the URL, but it can still be read on the network if HTTPS is not used.

401 name is confusing

401 Unauthorized actually means unauthenticated or invalid authentication. The name is historically confusing.

Sessions can be stored in many places

Session data can be stored in memory, Redis, database, or distributed session storage. In stateless JWT-based systems, session-like information may be stored in signed tokens.

UDP can support reliable protocols

UDP itself is unreliable, but applications can build reliability on top of it. QUIC is a common modern example.

DNS results may be cached

DNS lookup may not happen every time you open a website because browser, OS, router, or ISP may cache results.

NAT can cause inbound connection problems

Devices behind NAT are easy to connect outward but harder to connect inward from the internet unless port forwarding or traversal techniques are used.

Load balancers can work at different layers

Layer 4 load balancers use TCP/UDP information. Layer 7 load balancers understand HTTP-level data such as path, host, and headers.

TLS certificate validates identity, not business honesty

HTTPS tells you the connection is encrypted and the domain identity is verified. It does not guarantee that the company or website is morally trustworthy.

11. How to Explain in Interview

Computer networking is about how devices communicate using protocols. For web applications, DNS first converts a domain name to an IP address. Then the browser creates a TCP connection with the server. If the site uses HTTPS, a TLS handshake establishes encryption and verifies the server certificate. After that, HTTP requests and responses are exchanged. Cookies and sessions help maintain user state because HTTP itself is stateless. At scale, load balancers and reverse proxies distribute and manage traffic across backend servers.

12. Quick Revision Notes

Key definitions

- HTTP: Protocol used for web communication.
- HTTPS: HTTP secured with TLS.
- TCP: Reliable, ordered, connection-based transport protocol.
- UDP: Fast, connectionless transport protocol without delivery guarantee.
- DNS: Converts domain names to IP addresses.
- NAT: Translates private IPs to public IPs.
- Subnetting: Divides a network into smaller networks.
- Socket: Communication endpoint identified by IP and port.
- TLS: Security protocol for encrypted communication.
- Load balancer: Distributes traffic across servers.
- Reverse proxy: Forwards client requests to internal servers.

Important points

- HTTP is stateless.
- HTTPS uses TLS.
- TCP uses a 3-way handshake.
- TCP reliability comes from ACKs, sequence numbers, retransmission, and control mechanisms.
- UDP is useful when low latency matters more than perfect reliability.

- Cookies are client-side; sessions are usually server-side.
- 401 means authentication problem; 403 means permission problem.
- DNS lookup can be cached.
- NAT allows many private devices to share one public IP.

Common comparisons

- HTTP vs HTTPS
- GET vs POST
- 401 vs 403
- Cookie vs Session
- TCP vs UDP
- Load Balancer vs Reverse Proxy

Must-remember facts

- HTTP port: 80
- HTTPS port: 443
- DNS port: 53
- TCP handshake: SYN, SYN-ACK, ACK
- Private IP ranges include:
 - 10.0.0.0/8
 - 172.16.0.0/12
 - 192.168.0.0/16

Interview traps

- POST is not secure without HTTPS.
- HTTPS does not mean the website cannot be malicious.
- UDP is not useless.
- TCP is reliable but not magical.
- Sessions are not necessarily stored only in server memory.
- A reverse proxy can also do load balancing.

13. Practice Tasks

Task 1: Trace a URL request

Explain step by step what happens when you open:

```
https://www.amazon.com
```

Include DNS, TCP, TLS, HTTP, cookies, and load balancer.

Task 2: Draw TCP handshake

Draw the TCP 3-way handshake and explain why each step is needed.

Task 3: Classify protocols

Classify the following as mostly TCP or UDP:

- Web browsing
- DNS lookup
- SSH
- Online gaming
- Video call
- File download

Task 4: Design simple login flow

Explain how login works using cookies and sessions:

1. User submits username/password.
2. Server validates credentials.
3. Server creates session.
4. Browser stores cookie.
5. Future requests include cookie.

Task 5: Subnet calculation

For **192.168.1.0/24**, identify:

- Network address
- Broadcast address
- Number of total addresses
- Number of usable host addresses

Task 6: Compare status codes

Give examples where an API should return:

- **200**
- **201**
- **400**
- **401**
- **403**
- **404**
- **500**

Task 7: Write a tiny socket program

Write a simple TCP client-server program in Python where:

- Server listens on a port.
- Client connects to server.
- Client sends a message.
- Server replies with a message.

Example outline:

```
# Server idea
# 1. create socket
# 2. bind to host and port
# 3. listen
# 4. accept client
# 5. receive data
# 6. send response
```

Task 8: Explain NAT

Draw a home network with three devices and explain how all devices access the internet using one public IP.

Task 9: TLS explanation

Explain why TLS uses both asymmetric and symmetric encryption.

Task 10: Load balancer design

Design a system with three backend servers behind a load balancer. Explain:

- How traffic is distributed
- What happens if one server fails
- Why health checks are useful

14. Final Cheat Sheet

Core definition

Computer networks allow devices to communicate using protocols such as DNS, TCP, UDP, HTTP, HTTPS, and TLS.

Why it matters

Every web app, mobile app, backend service, database connection, and distributed system depends on networking.

Most asked questions

- What happens when you open a URL?
- HTTP vs HTTPS
- GET vs POST
- 401 vs 403
- Cookie vs session
- TCP vs UDP
- Explain TCP 3-way handshake
- Why is TCP reliable?
- What is DNS?

- What is NAT?
- What is subnetting?
- What is a socket?
- Explain TLS handshake
- Load balancer vs reverse proxy

Common comparisons

Comparison	One-line difference
HTTP vs HTTPS	HTTPS is HTTP secured using TLS.
GET vs POST	GET retrieves data; POST submits or creates data.
401 vs 403	401 means not authenticated; 403 means not authorized.
Cookie vs Session	Cookie is client-side; session is usually server-side.
TCP vs UDP	TCP is reliable and ordered; UDP is fast and connectionless.
Load Balancer vs Reverse Proxy	Load balancer distributes traffic; reverse proxy forwards and manages requests.

One-line interview answer

Computer networks define how devices communicate; in web systems, DNS finds the server, TCP creates reliable transport, TLS secures the connection, HTTP exchanges requests and responses, and components like cookies, sessions, NAT, subnets, load balancers, and reverse proxies make the system usable, scalable, and secure.